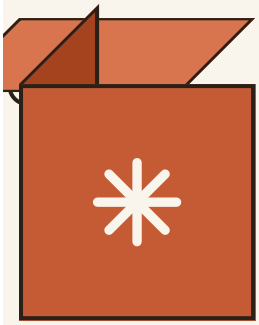
AI PARA TODO GUÍA



Los Mods de Claude

3 usos de los que nadie habla. Los prompts para que Claude te los construya solo, y cómo no meterte un troyano.

Alex Silvestre
@silvesttre.ia · s4sf.net/aiparatodo

GUÍA GRATUITA

QUÉ HAY DENTRO

01 Qué es un Mod (y por qué no es un plugin más)

02 Enciéndelos y mira cuáles ya tienes gratis

03 Uso 1 · El Mod que protege tus límites

04 Uso 2 · El panel en vivo de tus agentes

05 Uso 3 · El guardarraíl que frena un desastre

06 El truco de verdad: que Claude te lo construya

07 Antes de instalar uno: no te metas un troyano

08 Prompts para copiar y pegar

Llevas usando Claude Code para que escriba y arregle cosas. Un **Mod** es el siguiente nivel: le cambias a Claude Code la cara y el comportamiento. Y la parte buena es que **no hace falta programar**: se lo describes y Claude te lo construye.

Qué es un Mod (y por qué no es un plugin más)

Un Mod es un trozo de código (JavaScript o TypeScript) que corre **dentro** de Claude Code. Claude Code lo llama cuando pasa algo —usas una herramienta, envías un prompt, se dibuja la pantalla— y tu Mod puede mirarlo, cambiarlo o quedárselo. Eso le deja hacer tres cosas que ni las skills, ni los hooks, ni los MCP pueden:

- **Dibujar interfaz de verdad**: un panel al lado de la conversación o una banda encima del cuadro de texto, con botones y campos.
- **Meterse en una llamada a una herramienta**: frenar un comando peligroso y preguntarte, responder sin ejecutarlo, o mandar una petición a otro modelo.
- **Añadir un comando tuyo** (`/loquesea`) que lanza tu función al instante, sin gastar un turno de Claude, incluso mientras Claude trabaja.

Lo primero de todo

Los Mods necesitan Claude Code **v2.1.287 o superior** y vienen **encendidos de serie**. Comprueba tu versión con `claude --version` y actualiza si la tuya es más vieja.

Enciéndelos y mira cuáles ya tienes gratis

Algunas funciones de Claude Code **ya son Mods**. El propio `/diff` lo es. Para ver cuáles tienes cargados, escribe `/plugin` y mira la línea de debajo de las pestañas: te dice el número y los nombres, como `1 mod active · first-mod`.

En tu terminal

```
claude --version
/plugin
```

Comprueba la versión y mira los Mods que ya tienes cargados.

Hay uno gratis que mucha gente no conoce: **«You should know»**, un agente que vigila por detrás mientras Claude trabaja en algo largo y te avisa de lo que a ti o a Claude se os podría escapar. Se enciende así:

Activar un Mod que ya viene incluido

```
/plugin enable cc-plugin-you-should-know@builtin
```

Un vigilante que te avisa de cosas que se te escaparían. Viene apagado de serie.

Y Anthropic publica Mods de ejemplo en abierto (repo `anthropics/claude-code-playground`): **token-weather** (dibuja cómo de lleno va tu contexto), **blast-radius** (frena comandos peligrosos como `rm -rf` y te enseña qué harían) y **replay-theater** (un comando `/replay` que repasa los cambios de la última tanda).

Uso 1 · El Mod que protege tus límites

Lo simple no debería gastarte el modelo potente. Arreglar una errata o renombrar unos archivos no necesita a Opus: con Haiku va sobrado, y **así tus límites de Opus aguantan para lo que de verdad importa** (planear un lanzamiento, un error difícil). Un Mod puede mirar la tarea y mandar lo pequeño a un modelo barato por su cuenta, sin que tú cambies de modelo a mano.

PROMPT **Pídeselo a Claude (copia y pega)**

Constrúyeme un mod para Claude Code que, para trabajos pequeños (renombrar archivos, arreglar una errata, resumir un texto), use el modelo Haiku en vez del potente, para no gastar mis límites de Opus en tonterías. Añádele un comando /modelo que me diga qué modelo está usando ahora mismo.

Lo que hay por debajo (ejemplo real de la documentación)

```
on('command.run', { command: 'triage' }, async ($, e) => {
  const r = await $.model.complete({
    model: 'haiku',
    system: 'Responde con una palabra: bug, mejora o duda.',
    prompt: e.args,
    maxTokens: 20,
    timeoutMs: 15000,
  })
  const etiqueta = r.isAnswered ? r.text.trim() : 'desconocido'
  return { text: 'Etiqueta: ' + etiqueta }
})
```

Así manda un Mod un trabajillo a Haiku sin tocar tu conversación: $.model.complete con model: "haiku".

Por qué nadie lo cuenta

Todo el mundo cambia de modelo a mano con `/model` y se olvida. Un Mod lo hace solo, cada vez, para siempre. Es la diferencia entre acordarte… y que no haga falta acordarse.

Uso 2 · El panel en vivo de tus agentes

Cuando lanzas varios agentes a la vez, pierdes el hilo de qué está haciendo cada uno. Un Mod puede abrir un **panel en vivo** al lado de la conversación y pintar lo que pasa en tiempo real: cuántas herramientas lleva, qué está tocando, cuántas peticiones van. Tú solo se lo pides.

PROMPT **Pídeselo a Claude (copia y pega)**

Constrúyeme un mod que muestre en vivo qué están haciendo mis agentes de Claude Code: un panel al lado de la conversación con cada agente, qué herramienta está usando y cuántas peticiones lleva. Que se actualice solo mientras trabajan.

La pieza clave (ejemplo real de la documentación)

```
let herramientas = 0

export function register(on) {
  // Cada vez que Claude va a usar una herramienta
  on('tool.call', async ($, e, next) => {
    herramientas += 1
    $.ui.invalidate('ui.render')   // vuelve a dibujar
    return next(e)                  // deja que la herramienta corra
  })

  // Cada vez que Claude Code dibuja el spinner
  on('ui.render', { component: 'Spinner' }, async ($, e, next) => {
    return next({ ...e, props: { ...e.props, suffix: ' · herramientas: ' + herramientas +
'…' } })
  })
}
```

Un Mod cuenta cada herramienta que usa Claude y lo pinta junto al spinner: «Pensando · herramientas: 3…». Misma idea, a lo grande, para el panel.

Dónde se ve

El panel se dibuja en la terminal y en la pestaña Code de la app de escritorio. En la extensión de VS Code y en `claude -p` el Mod funciona igual, pero no pinta panel.

Uso 3 · El guardarraíl que frena un desastre

Un `rm -rf` o un `git push --force` en el sitio equivocado te arruina el día. Un Mod puede **retener** el comando peligroso antes de que corra, enseñártelo y preguntarte si tiras p'alante o lo cancelas. Es, de hecho, lo que hace el ejemplo oficial `blast-radius`.

PROMPT **Pídeselo a Claude (copia y pega)**

Constrúyeme un mod que, antes de ejecutar un comando peligroso (rm -rf, git reset --hard, git push --force), lo frene y me pregunte si quiero seguir o cancelar, enseñándome el comando. Para el resto de comandos, que no moleste.

Cómo frena y pregunta (ejemplo real de la documentación)

```
const RIESGO = /rm\s+-rf|git\s+reset\s+--hard|git\s+push.*--force/

on('tool.call', { tool: 'Bash' }, async ($, e, next) => {
  if (!RIESGO.test(e.command)) return next(e)   // lo normal, sin molestar
  let respuesta
  try {
    respuesta = await $.ui.ask('¿Ejecuto esto? ' + e.command, ['Ejecútalo', 'Cancela'])
  } catch {
    return { deny: 'El usuario no lo aprobó.' }
  }
  if (respuesta !== 'Ejecútalo') return { deny: 'Cancelado por el usuario.' }
  return next(e)   // sigue, y aún pasa el permiso normal
})
```

El Mod retiene la llamada, te pregunta con $.ui.ask y solo sigue si dices que sí.

Un guardarraíl no es una jaula

El Mod añade una pregunta tuya, pero no sustituye a los permisos de Claude Code: tras decir «Ejecútalo», el permiso normal sigue pidiéndose. Es una red de seguridad, no un cheque en blanco.

El truco de verdad: que Claude te lo construya

No tienes que escribir ni una línea. En una sesión de Claude Code, describes el Mod en tu idioma y Claude lo escribe, usando una skill que ya trae de serie (`plugin-authoring`). Lo guarda en una carpeta suya, te pide aprobar cada archivo, y lo carga en caliente: cada vez que le pides un cambio, se recarga al terminar y lo pruebas al momento.

1. **Pídelo** En Claude Code: «constrúyeme un mod que…» y lo que quieras. Claude carga la skill `plugin-authoring` y escribe los archivos.
2. **Aprueba** Te pregunta si activas la recarga en caliente para la sesión. Dile que sí: así se carga y se actualiza solo con cada cambio.
3. **Compruébalo** Escribe `/plugin` → pestaña Installed: ahí sale tu Mod y puedes apagarlo.
4. **Guárdalo** Un Mod que te escribe Claude vive solo en esa sesión. Para quedártelo, cópialo a una carpeta tuya (p. ej. `~/mods/mi-mod`) y ábrelo con `claude --plugin-dir ~/mods/mi-mod`.

Cargar un Mod tuyo en una sesión

```
claude --plugin-dir ./mi-mod
```

Sin instalarlo: lo carga solo para esa sesión y lo recarga cuando cambias un archivo.

Antes de instalar uno: no te metas un troyano

Esto es lo más importante de toda la guía. Un Mod **corre con tus permisos y no está aislado**. Una vez cargado puede:

- Leer y escribir tus archivos, arrancar programas y hacer peticiones a internet.
- Leer tus secretos: variables de entorno y ajustes, **incluida una clave de API** que tengas ahí.
- Ver toda tu sesión: cada prompt que envías y cada herramienta que usa Claude.
- Actuar sin preguntarte y gastar tu plan o tu clave llamando a un modelo.

Por eso puedes **auditar un Mod antes de instalarlo**, sin ejecutarlo. Clona sus archivos y pásale el validador: te lista qué eventos engancha (`hooks:`) y qué le pide a Claude Code hacer (`calls:`), como leer un archivo o salir a la red.

Auditar un Mod sin ejecutarlo

```
claude plugin validate ./un-mod
```

Mira las líneas hooks: y calls: de la salida. Si pide cosas que no cuadran con lo que dice hacer, no lo instales.

La regla de oro

Instala Mods solo de gente y de sitios en los que confíes. **Si no sabes quién lo hizo, no lo instales.** ¿Un apuro? Arranca con `claude --safe-mode` y esa sesión va sin ningún Mod.

Prompts para copiar y pegar

Pégalos tal cual en una sesión de Claude Code (en un proyecto de confianza) y deja que te los construya. Empieza por uno.

PROMPT **El medidor de contexto**

Constrúyeme un mod que me dibuje encima del cuadro de texto cómo de lleno va mi contexto después de cada respuesta, en porcentaje y con una barra, y que me avise cuando pase del 80%.

PROMPT **El parte del día**

Constrúyeme un mod con un comando /parte que me resuma en 5 líneas qué hemos cambiado hoy en este proyecto (archivos tocados y por qué), para pegarlo en mi registro diario.

PROMPT **El vigilante de claves**

Constrúyeme un mod que, antes de que Claude escriba en un archivo, revise si se está colando una clave de API o una contraseña, y si la ve, lo frene y me avise.

PROMPT **El cronómetro de tareas**

Constrúyeme un mod que mida cuánto tarda cada tarea (desde que envío el prompt hasta que Claude termina) y lo muestre junto al spinner, para saber qué me cuesta más.

Lo que te llevas

- Un Mod es código que corre DENTRO de Claude Code, con tus permisos, y le añade paneles, comandos y reglas a tu medida.
- No programas: se lo pides a Claude y te lo construye, lo apruebas y lo pruebas al momento.
- Los 3 usos que cambian el día: enrutar lo simple a un modelo barato, un panel en vivo de tus agentes y un guardarraíl que frena desastres.
- Audita siempre con `claude plugin validate` antes de instalar uno de fuera. Si no sabes quién lo hizo, no lo instales.
- Necesitas Claude Code v2.1.287+, y ya vienen encendidos.

Por dónde empezar

Hoy	Comprueba tu versión, abre `/plugin` y enciende «You should know».
Esta semana	Pídele a Claude el Mod del Uso 1 (enrutar lo simple a Haiku) y pruébalo un par de días.
Cuando te sientas cómodo	El panel de agentes y el guardarraíl. Guárdate los que te sirvan en `~/mods`.
Antes de instalar de fuera	Siempre `claude plugin validate`. Confía en el autor o no lo instales.

FUENTES OFICIALES

Mods — visión general (documentación oficial)
`https://code.claude.com/docs/en/plugins/mods/overview`

Crear un Mod (pedírselo a Claude o escribirlo tú)

`https://code.claude.com/docs/en/plugins/mods/create`

Mods de ejemplo de Anthropic

`https://github.com/anthropics/claude-code-playground/tree/main/claude-code/mods`

Hecho con Claude · **Comparte esto con quien se pelea con la IA.**
s4sf.net/aiparatodo